

ETX 融合反超方案

作者：Kailun Chen · 状态：v1 计划 · 日期：2026-09-25 · 依据：examples/llm/results/bench_2026-09-25.log、docs/ETX_Technical_Design.md 第 15.4–15.6 节

0. 结论先行

- **目标 1（必须）**：同一套 tile，ETX 融合版比”每个算子一个 kernel 加 HIP Graph”快 15% 以上。现在慢 6–15%。
- **目标 2**：三个 Qwen 模型上超过 vLLM 最强配置 15% 以上。现在慢 2 倍。
- **目标 3**：全程保持逐 token 与 HF greedy 一致，输出可复现。
- **路线**：先让屏障变便宜（每层全卡屏障从 8 次降到 2 次），再让屏障有事可做（跨屏障预取权重），最后把 tile 提到 3 TB/s 以上。前两步是编译器和运行时的工作，是 ETX 的价值所在；第三步是 tile 工程，决定和 vLLM 的差距。
- **预算**：7 个阶段，约 10 次 GPU 会话，每次 1–2 小时，1x MI300X 每小时 \$2.99，合计约 \$40–60。现有余额 \$26.20，需要补充。

1. 现状与根因

同一台 MI300X，1024 上下文，batch 1（2026-09-25）：

模型	vLLM 最强	ETX 不融合（同 tile，HIP Graph）	ETX 融合	手写 fleet
Qwen2.5-1.5B	1.86 ms	4.05	4.47	—
Qwen3-8B	4.91 ms	9.45	10.07	—
Qwen3-30B-A3B	4.84 ms	9.07	10.48	—
DeepSeek-V2-Lite	4.12 ms	未测	3.78（fleet 的 tile）	3.57

Qwen3-8B 逐阶段 trace，每层 274 μ s，带宽下限约 98 μ s：

去向	时间	根因
4 个 GEMV 阶段	190 μ s	有效带宽 1.5–2.5 TB/s，任务小、在途读请求不足
attention 加 merge	63 μ s	按位置分块后线程串行点积；merge 串行读 64 个分块
8 次全卡屏障	约 9 μ s 多于 kernel 边界	300 个 worker 轮询一个计数器，最后到达者回写，8 个 XCD 各自失效缓存

三个根因：1. **图的形状错了**。我给 Qwen 建的图每层 8 次全卡屏障。fleet 的图每层只有 2 次，其余 6 次是 XCD 内部事件。ETX 最有价值的能力是按 chiplet 建图，Qwen 的图没有用。2. **屏障比 kernel 边界贵**。一次约多 2 μ s。batch=1 的 decode 是一条”每环都要上一环完整输出”的链，屏障次数是结构决定的，只能让每次变便宜，或者让等待的时间有事可做。3. **tile 差**。GEMV 和 attention 都被访存延迟卡住。这和融合无关，但决定了和 vLLM 的距离。

2. 总体设计：把每层做成”每个 XCD 一片”

fleet 的做法可以推广成编译器规则。一层的计算按 XCD 切成 8 片，片内只用本地事件，片与片之间只在两处汇合：

XCD k 内部（本地事件，L1 失效即可）

$x \xrightarrow{\text{norm}} \text{qkv 的第 } k \text{ 片行} \rightarrow \text{本 XCD 的 KV 头组 attention} \rightarrow \text{merge} \rightarrow \text{o_proj 的第 } k \text{ 段 K}$
| 写本 XCD 的部分和
 o_k

$x + \sum \text{o_k} \xrightarrow{\text{norm}} \text{gate_up 的第 } k \text{ 片行} \rightarrow \text{down 的第 } k \text{ 段 K} \rightarrow \text{写本 XCD 的部分和 } d_k$
| 全卡屏障 2: 8 个部分和就绪

全卡屏障 1: 8 个部分和就绪
 $\text{下一层 qkv 的前置: } x = x + \sum d_k \text{ (读 8 个向量, 比一次屏障便宜)}$

要点：- 行切分给 **XCD**，K 切分给 **XCD**。qkv、gate_up 按输出行切片，每片正好是本 XCD 后续需要的部分；o_proj、down 按 K 切片，每个 XCD 只用自己片内产生的输入，输出是部分和。- **部分和的求和放进下一阶段的前置**。这就是 fleet 的 FOLD_PARTIALS，也是设计文档里的“重算换事件”。求和读 $8 \times H$ 个浮点，远比一次全卡屏障便宜。- **attention 按 KV 头组分给 XCD**。GQA 下每个 XCD 拿到 $NKV/8$ 个 KV 头和它们的 query 头，KV cache 的读写都在本 XCD 内。- **权重布局按片重排**。打包时把每个 XCD 的行片和 K 片放成连续块，任务读的是连续内存。

每层全卡屏障从 8 次降到 2 次，其余同步都是 XCD 内部的本地事件，成本约 $1 \mu s$ 且不需要 L2 失效。

3. 分阶段计划

阶段 0：补齐测量（1 次会话）

- DeepSeek 上补测“fleet 的 tile 不融合”。这一档决定“本地事件的图能否让融合赚钱”的结论。
- 屏障微基准：304 个 workgroup 下，ETX 全卡屏障、两级屏障、HIP Graph kernel 边界各多少微秒。把数字写进 gfx942.yaml 的成本表。
- 验收：三个数字进表，DeepSeek 不融合的数据进对比表。

阶段 1：编译器自动生成 XCD 本地图（2 次会话）

- **P2 扩展“按域切片”**：一个 grid 的输出被下游按域消费时，把生产者按域切片放置，事件降为 DOMAIN 作用域。输入是 edge map，比如 qkv 行块到 KV 头组的映射 “ $i \rightarrow (i/\text{rows_per_group})$ ”。
- **构图器改成分片图**：examples/llm/model.py 按第 2 节生成 grid 和事件；部分和缓冲区 [8][H]；前置求和作为 Pass 3 的重算前置。
- **权重打包按片重排**：prep.py 输出片连续的布局，manifest 记录片的偏移。
- **验收**：三个 Qwen 模型逐 token 仍与 HF 一致；每层全卡屏障 2 次；融合版至少追平不融合版。

阶段 2：两级全卡屏障（1 次会话）

- 到达分两级：先到本 XCD 的计数器，XCD 的最后到达者做一次 L2 回写并到达全卡计数器。这一半已经有了。
- 等待也分两级：每个 XCD 只有一个 waiter 轮询全卡计数器，其余 worker 轮询本 XCD 的本地字。waiter 看到完成后先做本 XCD 的 L2 失效，再写本地字；其余 worker 只做 L1 失效。这和之前被证明不安全的 relay 不同：失效由本 XCD 内的 worker 做，并且 waiter 在写本地字前用 s_waitcnt 等失效完成，其他 worker 读到本地字后再各自做 L1 失效。两级方案要先用 Qwen2.5 的非确定性测试验证，通过才启用。
- **验收**：屏障成本降到 $1 \mu s$ 附近；Qwen2.5 三次运行输出一致。

阶段 3：跨屏障预取权重（2 次会话）

这是独立 kernel 结构上做不到事，也是融合真正反超的来源。

- worker 的下一个任务在静态队列里是已知的，权重地址在等待前就能算出。等待循环里穿插发出下一任务权重块的读请求，目标是 L2（用 touch 读）或直接进 LDS 暂存区。
- LDS 预算：dense 模型每个 CU 一个 workgroup，64 KB 里输入向量占 16–48 KB，剩余部分做权重暂存；不够时退回 L2 预热。
- 这就是设计文档的 Pass 7，机器模型里的 `async_copy_to_lds` 能力在 gfx942 上标为 none，改成“普通读进 LDS”的降级实现。
- 验收：屏障后的 GEMV 起步时间（trace 里的 first-ready 到 body 开始）明显缩短；融合版比不融合版快 15% 以上。

阶段 4：合并小阶段（1 次会话）

- post（q/k norm 加 RoPE 加写 KV）并进 qkv 的收尾：让 qkv 的每个任务正好覆盖整数个头的行，收尾直接做。
- merge 由每个头最后完成的 attention 任务顺手做（最后到达者模式，运行时已支持）。
- 验收：每层阶段数从 8 降到 5；这两处的 30–40 μ s 消失。

阶段 5：tile 提速到 3 TB/s 以上（2–3 次会话）

- **GEMV**：输入向量改成 bf16 存 LDS，占用减半，让 dense 模型也能每 CU 放两个 workgroup；每个任务的权重块提到 256 KB 以上；在途读请求 32 个；non-temporal 读取默认开。目标 3.2 TB/s，参照 vLLM 在 8B 上的 3.0 TB/s。
- **attention**：改成 4 个 wave 各管一段位置、按 128 维向量化点积的 flash-decode 写法，输出部分和；1024 上下文下每层目标 5 μ s 以内。
- **lm_head**：已在 3 TB/s，不动。
- 另一条路是把 vLLM 或 AITER 的 GEMV 和 attention 作为 tile 链进来。ETX 的链接模式接受任何 HIP 函数，但它们是整 kernel 形态，要拆成 tile 才能用，工作量不小，作为备选。
- 验收：Qwen3-8B 融合版 6 ms 以内。

阶段 6：对比与交付（1 次会话）

- 四个模型 $\times$ {vLLM 最强、不融合、融合}，1024 上下文，各 3 轮，全部逐 token 对 HF。
- 更新设计文档、精简版 PDF、README；把屏障成本和预取收益写进机器模型的成本表。

4. 预期收益（Qwen3-8B，每 token）

阶段	融合版预计	依据
现在	10.07 ms	实测
阶段 1，屏障 8→2	约 9.3 ms	每层省 6 次约 5 μ s 的屏障加等待，36 层
阶段 2，两级屏障	约 9.2 ms	剩余 2 次各省 1 μ s
阶段 3，预取	约 8.3 ms	每次屏障后 GEMV 的起步延迟约 10 μ s 被隐藏
阶段 4，合并小阶段	约 7.2 ms	每层省 30 μ s
阶段 5，tile 3.2 TB/s	约 5.5 ms	$15.1\text{ GB} \div 3.2\text{ TB/s}$ 加剩余同步

不融合版在阶段 5 后预计 6.3 ms，vLLM 4.91 ms。所以：融合反超不融合 15% 靠阶段 1–4；追上 vLLM 靠阶段 5；超过 vLLM 15% 需要 tile 到 3.6 TB/s 以上或 MoE 上更大的路由收益。MoE 模型上预期更好：128 个专家选 8 个，专家权重的读取可以在路由确定后立刻预取，独立 kernel 做不到。

5. 风险

- **两级屏障的正确性。** relay 的分层 acquire 已被证明不安全，新方案必须过 Qwen2.5 的确定性测试，不通过就只做到达两级、等待仍各自轮询。
- **预取占用 LDS 和寄存器。** 可能压低 GEMV 的在途请求数，需要在 P1 的资源账里统一分配，A/B 决定深度。
- **分片图对模型结构的假设。** 头数不能被 8 整除、专家数不能被 8 整除时要退化成不均匀切片，构图器要处理余数。
- **tile 提速的不确定性最大。** 3.2 TB/s 是估计，达不到时 ETX 相对 vLLM 仍会落后，但目标 1 不受影响。
- **GPU 可用性。** Hot Aisle 近两天经常没有 1x 或 2x 空位，每次要等几小时。

6. 阶段 0–1 的实测结果（2026-09-25）

分片图已实现（examples/llm/model_sliced.py）：每层 2 次整卡事件，其余本地；部分和由每个 XCD 的一个 fold 任务求和一次；三个模型全部 32/32 与 HF 一致、三次运行一致。同一套 tile 下，融合版从落后不融合版 6–25% 收到 2–5% 以内，但没有反超；逐算子的不融合版仍是最快的一档。路上修掉三处 tile 缺陷（逐任务求和重复读 147 KB、求和寄存器块溢出、attention 每任务一个头），并发现每 CU 两个 workgroup 会让轮询晚 8–14 μ s，默认改为一个。L2 预热式预取是负收益。DeepSeek 的“fleet tile 不融合”无法测：fleet 的 tile 依赖持久化 kernel。底线未达成，按第 6 节的决策点转向 MoE 路由、多卡 and tile 质量。详见设计文档 15.7。

7. 顺序和取舍

先做阶段 0 和 1。阶段 0 只要 20 分钟，直接回答“本地事件的图能不能让融合赚钱”；阶段 1 是所有后续工作的基础，也是编译器里最有价值的一条规则。这两步做完，如果融合版仍不能追平不融合版，就应该重新考虑 batch=1 是否是 megakernel 的正确场景，把重心转向 MoE 路由和多卡这些独立 kernel 做不到的地方。